Fruit Vision

CNN-Based Fruit Classification System

SOFTWARE DESIGN DOCUMENT

Supplementary Chapters

Chapter 6 — System Context
Chapter 7 — Implementation
Chapter 8 — Testing

Chapter 6
System Context

6.1 Introduction

This chapter defines the boundary of the Fruit Vision system and describes how it fits into its surrounding environment. It identifies the external entities that interact with the system, the information exchanged across the system boundary, and the operating environment in which the software runs. Establishing this context ensures that the design decisions in the following chapters are grounded in a clear understanding of who uses the system and what it depends upon.

6.2 System Context Overview

Fruit Vision is a self-contained CNN-based classification system exposed to the outside world through a single web interface. From an external point of view, the system behaves as one logical unit: a user submits a fruit image and the system returns a predicted class together with a confidence score. Internally the system relies on two supporting data stores — the trained CNN model artifact and the labelled image dataset used to produce it — but neither is exposed directly to the user.

The only human actor is the **End User**, who can be a retailer, farmer, student, or any person wishing to identify a fruit from an image. All interaction takes place through a standard web browser, which keeps the system accessible and platform-independent.

6.3 Context Diagram

The context diagram in Figure 6.1 presents a Level 0 view of the system. It shows the End User as the sole external entity and the two internal data stores that the system reads from, along with the direction of each major data flow.

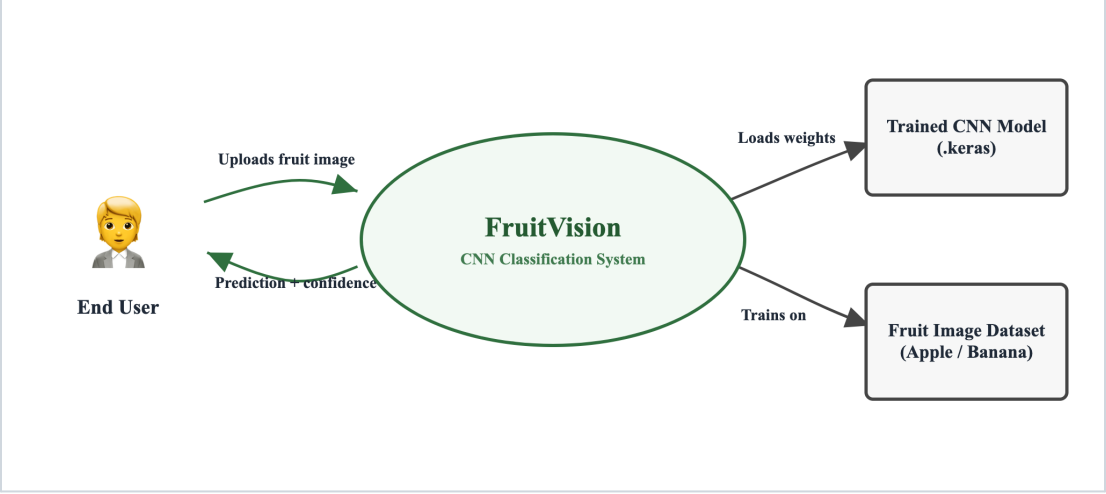


Figure 6.1 — FruitVision System Context Diagram (Level 0)

6.4 External Entities and Data Flows

Table 6.1 summarises the entities that cross the system boundary and the data that flows between them and the Fruit Vision system.

Entity / Store	Direction	Data Flow	Description
End User	Input	Fruit image (JPEG/PNG)	The user uploads an image through the web interface for classification.
End User	Output	Prediction & confidence	The system returns the predicted class name and a confidence percentage.
Trained CNN Model	Read	Model weights (.keras)	The backend loads the saved model to perform inference on the uploaded image.
Image Dataset	Read	Labelled images	Apple and Banana images used offline to train and validate the model.

Table 6.1 — External entities and data flows

6.5 Operating Environment

The system operates in a lightweight client–server environment. The client requires only a modern web browser, while the server hosts the REST API and the deep-learning runtime. Table 6.2 lists the environment on each side.

Tier	Requirement
Client	Any modern web browser (Chrome, Firefox, Edge, Safari) on desktop or mobile.
Server	Python 3.9+, FastAPI, Uvicorn, and TensorFlow/Keras for model inference.
Model	Trained Keras model artifact (fruitvision-latest.keras) with class metadata.
Network	HTTP connection between the browser and the backend REST API.

Table 6.2 — Operating environment

6.6 Assumptions and Dependencies

- The uploaded file is a valid image containing a single, clearly visible fruit.
- A trained model artifact is present on the server before predictions are requested.
- The backend service is running and reachable from the client browser.
- The system currently recognises two classes — Apple and Banana — and depends on the labelled dataset for any future expansion of classes.

Chapter 7
Implementation

7.1 Implementation Overview

The Fruit Vision system is implemented as a layered application in which each layer has a single, well-defined responsibility. A browser-based presentation layer collects the image, a FastAPI application layer exposes the REST endpoints, a preprocessing stage prepares the image, and a TensorFlow/Keras CNN performs the classification. Figure 7.1 illustrates how a request flows through these layers.

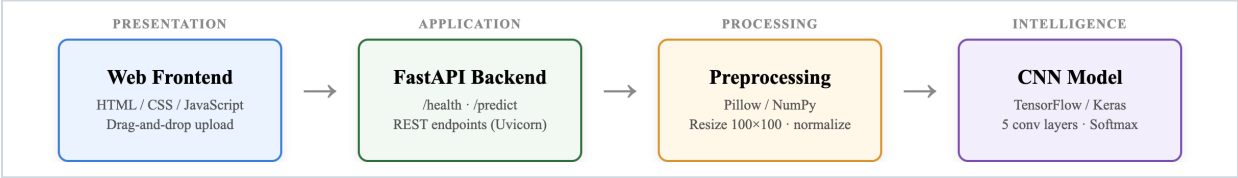


Figure 7.1 — FruitVision layered implementation architecture

7.2 Technology Stack

Each layer is built with a focused, widely-used technology, as summarised in Table 7.1.

Layer	Technology	Purpose
Presentation	HTML, CSS, JavaScript	Image upload UI and result display in the browser.
Application	FastAPI + Uvicorn	REST endpoints (/health, /predict) and request handling.
Processing	Pillow, NumPy	Decode, resize (100×100) and format the image for the model.
Intelligence	TensorFlow / Keras	Custom CNN performing the fruit classification.
Testing	pytest	Automated unit and integration tests.

Table 7.1 — Technology stack by layer

7.3 Project Structure

The source code is organised into clearly separated packages:

```
fruitvision/
|-- backend/            # FastAPI application (api.py)
|-- frontend/           # Web UI (index.html, styles.css, app.js)
|-- src/fruitvision/
|   |-- config.py       # Model & training configuration
|   |-- data/dataset.py # Dataset loading pipeline
|   |-- models/cnn.py   # CNN architecture
|   |-- utils/image.py  # Image preprocessing
|   |-- inference.py    # Model loading & prediction
|   `-- train.py        # Training entry point
|-- artifacts/          # Trained models, logs, metrics
`-- tests/              # Automated test suite
```

7.4 Module Implementation

7.4.1 CNN Model Architecture

The classifier is a custom Convolutional Neural Network built with the Keras Sequential API. It combines on-the-fly data augmentation, five convolutional layers with increasing filter depth (32 → 128), max-pooling, dropout regularisation, and a dense Softmax classification head.

```
model = models.Sequential([
    layers.Input(shape=input_shape),
    data_augmentation,                      # random flip / rotation / zoom
    layers.Rescaling(1.0 / 255),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPool2D(), layers.Dropout(0.25),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPool2D(), layers.Dropout(0.25),
    layers.Conv2D(128, (3, 3), activation="relu"),
    layers.MaxPool2D(), layers.Flatten(),
    layers.Dense(256, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation="softmax"),
])
```

7.4.2 Image Preprocessing

Every uploaded image is decoded, converted to RGB, and resized to the 100×100 input shape expected by the model before being wrapped into a single-item batch.

```
def load_image_from_bytes(image_bytes, config=CONFIG):
    with Image.open(BytesIO(image_bytes)) as img:
        img = img.convert("RGB")
        img = img.resize((config.img_width, config.img_height))
        return np.array(img, dtype="float32")
```

7.4.3 Inference Engine

The inference module lazily loads and caches the trained model, runs the forward pass, and maps the raw output vector to human-readable class names, returning the highest-scoring label as the final prediction.

```
def predict(image_batch, config=CONFIG, model_path=None):
    model = load_model(model_path=model_path)
    class_names = load_class_names()
    preds = model.predict(image_batch, verbose=0)[0]
    return {cls: float(score) for cls, score in zip(class_names, preds)}

def top_prediction(scores):
    return max(scores.items(), key=lambda item: item[1])
```

7.4.4 Backend REST API

The backend is a FastAPI application exposing a health-check endpoint and a prediction endpoint. The /predict endpoint validates that the upload is an image, preprocesses it, runs inference, and returns the prediction as JSON. FastAPI automatically generates interactive API documentation, shown in Figure 7.2.

```
@app.post("/predict")
async def predict_fruit(file: UploadFile = File(...)):
    if not file.content_type or not file.content_type.startswith("image/"):
        raise HTTPException(status_code=400, detail="Please upload an image file.")
    image_bytes = await file.read()
    image_batch, processed = preprocess_upload(image_bytes, CONFIG)
    scores = predict(image_batch, CONFIG)
    label, confidence = top_prediction(scores)
    return {"prediction": label, "confidence": confidence, "scores": scores}
```

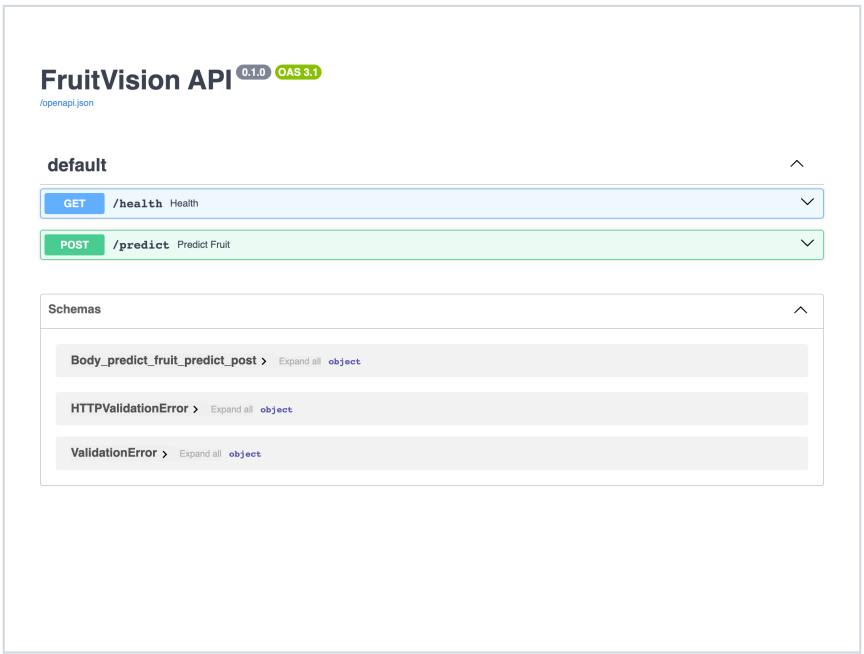


Figure 7.2 — Auto-generated FastAPI (Swagger) documentation

7.4.5 Web Frontend

The frontend is a single lightweight page built with HTML, CSS, and vanilla JavaScript. It lets the user choose an image, previews it, sends it to the /predict endpoint using the Fetch API, and renders the returned prediction and confidence. Figure 7.3 shows the landing page.

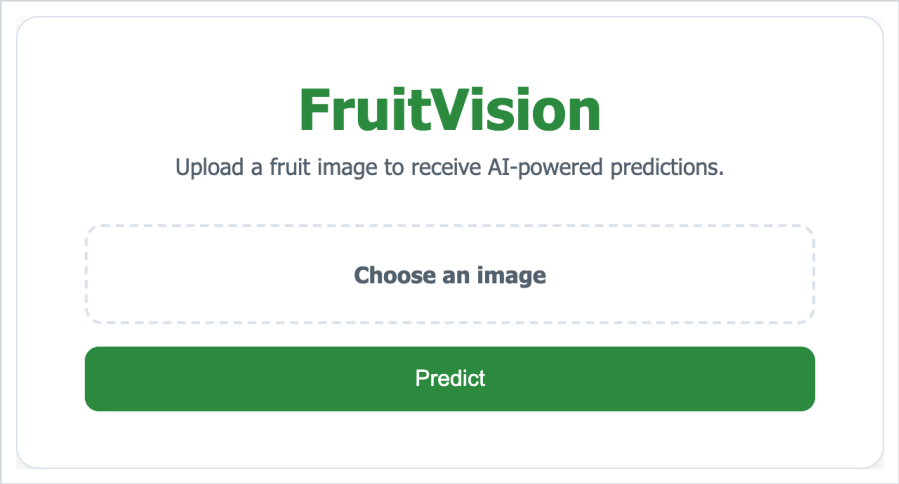


Figure 7.3 — Fruit Vision web interface (landing page)

7.5 Running the Application

The system is started by launching the backend API and serving the static frontend, after which the interface is available in any browser:

```
# 1. Start the FastAPI backend (serves the CNN model)
PYTHONPATH=src uvicorn backend.api:app --host 127.0.0.1 --port 8000

# 2. Serve the web frontend
cd frontend && python -m http.server 5500

# 3. Open http://127.0.0.1:5500 in a browser
```

7.6 Working Application

The following figures show the fully working system classifying real images. In Figure 7.4 an apple image is correctly identified as **Apple**, and in Figure 7.5 a banana image is correctly identified as **Banana**, each with its associated confidence score returned by the CNN.

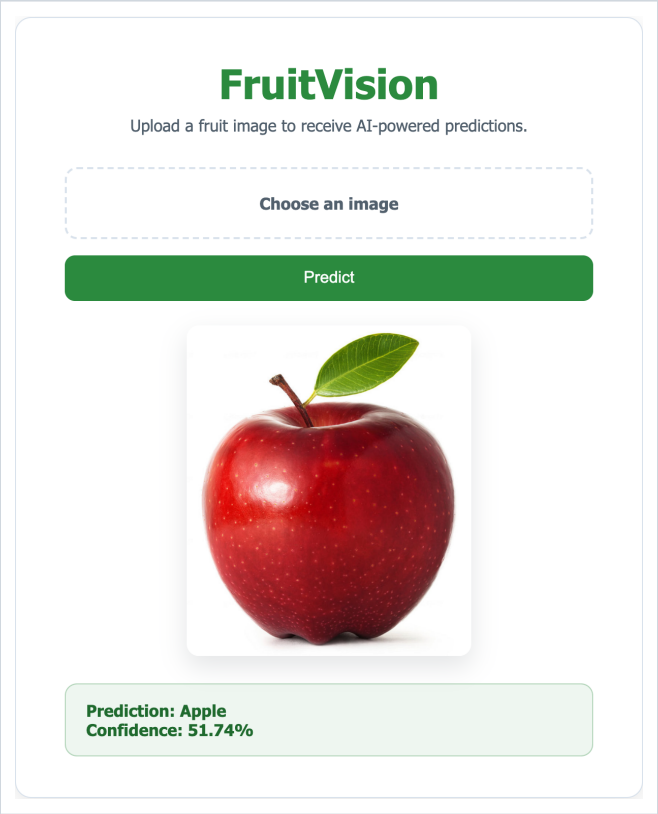


Figure 7.4 — Correct classification of an apple image

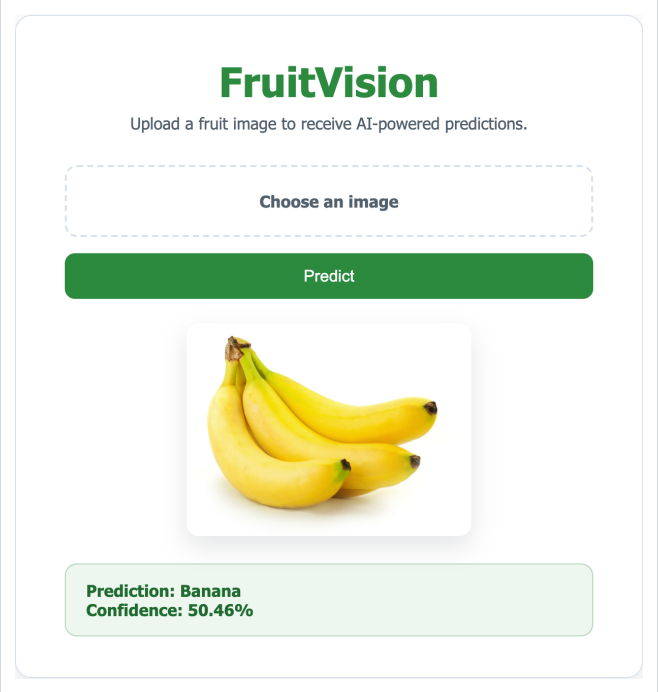


Figure 7.5 — Correct classification of a banana image

Chapter 8
Testing

8.1 Testing Overview

Testing verifies that each component of the Fruit Vision system behaves as expected and that the components work correctly together. A suite of automated tests, written with the pytest framework, exercises the image preprocessing pipeline, the prediction logic, and the REST API. Automating these checks allows the system to be re-validated quickly whenever the code or model changes.

8.2 Testing Strategy

- **Unit testing** — validates individual functions such as image resizing and the selection of the top-scoring class in isolation.
- **Integration testing** — exercises the FastAPI endpoints end-to-end, confirming the API, preprocessing, and model cooperate correctly.
- **Functional testing** — confirms that real fruit images produce the correct predicted class through the running application.

8.3 Test Environment

The tests were executed in the environment described in Table 8.1.

Component	Detail
Framework	pytest 8.4
Runtime	Python 3.9 with TensorFlow/Keras and FastAPI TestClient
Model	Trained artifact fruitvision-latest.keras (Apple / Banana)
Platform	macOS (darwin)

Table 8.1 — Test environment

8.4 Test Cases

Table 8.2 lists the test cases executed against the system together with their inputs, expected results, and outcome.

ID	Test Case	Input	Expected Result	Status
TC-01	Health endpoint	GET /health	Returns HTTP 200 with status "ok"	Pass
TC-02	Reject non-image upload	POST /predict with a text file	Returns HTTP 400 error	Pass

ID	Test Case	Input	Expected Result	Status
TC-03	Predict from image	POST /predict with a fruit image	Returns a class in {Apple, Banana} with a valid confidence	Pass
TC-04	Image resize	120×90 image	Array reshaped to 100×100×3 (float32)	Pass
TC-05	Batch dimension	Single image	Batch shape (1, 100, 100, 3)	Pass
TC-06	Top prediction	{Apple: 0.82, Banana: 0.18}	Selects "Apple"	Pass
TC-07	Close-score prediction	{Apple: 0.49, Banana: 0.51}	Selects "Banana"	Pass

Table 8.2 — Executed test cases

8.5 Test Execution Results

All seven automated tests were executed with a single pytest run. Every test passed, as shown in the execution output in Figure 8.1.



Figure 8.1 — pytest execution output (7 passed)

8.6 Test Summary

Total Tests	Passed	Failed	Pass Rate
7	7	0	100%

Table 8.3 — Test execution summary

The results confirm that the Fruit Vision system meets its functional requirements: images are validated and preprocessed correctly, the CNN returns predictions for supported fruit classes, and the REST API responds as specified. The complete pass rate provides confidence that the core classification pipeline is working reliably.